

CSCE 701 CYBERSECURITY · MODULE 08 · LECTURE NOTES

The Supply Chain

Securing what you did not write.

MODULE OUTCOMES

- Explain the dependency attack surface and its growth.
- Explain the build pipeline as a target.
- Explain provenance, signing, and reproducible builds.
- Assess a project's dependency risk.
- Design a response to a dependency advisory.

1 The dependency surface

YOUR DEPENDENCY TREE IS CODE YOU RUN WITH YOUR PRIVILEGES AND DID NOT REVIEW

A typical application has tens of direct dependencies and hundreds to several thousand transitively — and every one of them executes with your process's full privileges, with access to your memory, your filesystem, and your network position.

So the trust decision is not 'do I trust this library' but 'do I trust the entire transitive closure, and every person who can publish an update to any part of it' — which is a much larger and considerably less answerable question, and is not the question anyone asks when adding a package.

And install-time code execution makes it worse. A package whose install script runs is code executing on your build machine before any review, test, or human inspection — which is why install scripts are the single highest-risk feature in a package manager, and why disabling them is a cheap and effective control.

Which means the first control is reducing the tree. A dependency you removed cannot be compromised, and Module 01 §1's reduce-the-surface principle applies here more directly than anywhere else in the course — the question 'do we need this package for the three functions we use' has a measurable security answer.

Risk	The mechanism	The control
Known vulnerability	A published advisory against a version you are running, which you have not acted on.	Continuous scanning plus an actual patch cadence — the scanning is easy and the cadence is the hard part.

Risk	The mechanism	The control
Abandoned package	Nobody will fix the next vulnerability, because nobody is maintaining it.	Check maintenance activity before adopting, and treat abandonment as a reason to migrate rather than a neutral fact.
Maintainer handover	A new publisher inherits the trust you placed in the previous one, including the right to push to every system that depends on it.	Pin versions, review diffs on upgrade, and watch for ownership changes — see the note.
Typosquatting and confusion	A near-identical package name installed by mistake, or a public name shadowing an internal one.	Lockfiles with hashes, scoped internal registries, and verifying the name on first adoption.
Build-time execution	Install and post-install scripts run on the build machine before anything is reviewed.	Disable install scripts where the ecosystem allows, and build in an ephemeral sandbox (§2).

The maintainer-handover row is the structurally hardest. The trust was placed in a person and the package manager mostly does not surface a change of publisher — so a transfer of a widely-depended-upon package to a new owner is invisible to the thousands of projects affected. **There is no good control available to a consumer beyond pinning and reading diffs**, which does not scale, and saying so is more useful than pretending otherwise.

2 The build

THE BUILD SYSTEM IS THE HIGHEST-VALUE TARGET IN MOST ORGANISATIONS

It holds credentials to the artefact registry, frequently to production, and to the source repository — and its output is trusted and deployed without further review, which is the whole point of a build system.

So compromising the build compromises everything it produces, for as long as nobody notices — which is a far better position for an attacker than compromising any single server, and is why this is where sophisticated attacks go.

And it is typically less hardened than production, because it is treated as developer infrastructure rather than as a production system that happens to hold production credentials — which is a categorisation error with large consequences.

So: treat the build as production. Least privilege on its credentials, ephemeral runners, no secrets in build logs, review of pipeline changes, and network segmentation from developer workstations (Module 04 §2) — and the segmentation matters because a phished developer should not reach the builder.

EPHEMERAL RUNNERS

a fresh machine or container per build, destroyed afterwards. A compromise does not persist into the next build, which removes the attacker's foothold.

SCOPED, SHORT-LIVED CREDENTIALS

the build receives a token for THIS artefact, valid for minutes -- not a standing deploy key with broad rights (Module 07 section 2's platform identity).

PINNED, VERIFIED INPUTS

lockfiles with content hashes, and pinned versions of the tools themselves. "latest" means "whatever was published since you last looked".

SEPARATE THE BUILD FROM THE RELEASE

building is untrusted work on untrusted input; signing and publishing is privileged. Different stages, different credentials, different machines.

AND REVIEW PIPELINE CHANGES LIKE CODE

a pipeline definition is code with production credentials, and it is frequently exempt from the review that ordinary code requires.

Separating build from release is the structural one. It means that compromising the build machine does not yield a signature — the attacker can produce a malicious artefact and cannot get it signed, so the signature verification at deploy time catches it. **Which converts a total compromise into a detected attempt**, and it is the highest-leverage change in this list.

3 Provenance

- **A software bill of materials.** Answers 'what is actually in this artefact' — which is precisely the question you cannot answer during an advisory response without one (see the callout).
- **Artefact signing.** Answers 'did this come from my build' — and **it is only as good as the key handling** (Module 07), so a signing key on the build machine provides much less than it appears to (§2).
- **Build attestation.** Answers 'what source revision and what build steps produced this artefact', signed by the build platform rather than by a human — which is what SLSA and similar frameworks specify.
- **Reproducible builds.** Answers 'can an independent party obtain byte-identical output from the same source' — **which makes a compromised build detectable by a third party rather than only by the builder.**
- **And an artefact registry you control**, so that what you deploy is what you built and verified rather than whatever a public registry currently serves under that name. **Reproducibility is the strongest of these, because it removes the need to trust the builder at all** — and it is the hardest, because timestamps, build paths, parallelism, and locale all leak into outputs and each has to be hunted down.

THE SBOM'S VALUE IS ENTIRELY IN THE RESPONSE, NOT THE DOCUMENT

An advisory lands on a widely-used library at nine in the morning. The question you must answer is which of your hundred services contain it, at which version, and which of those are exposed.

Without an inventory, that takes days of grepping and guesswork, during which you do not know your own exposure — which is exactly the position nearly every organisation found itself in during the major logging-library advisory, and the inventory gap was the story rather than the vulnerability.

With one, it is a query. And the value is the *query time* rather than the compliance artefact — which is why an SBOM generated at build time, filed, and never read is wasted effort that satisfies an auditor and helps nobody.

So the test is simple: can you answer 'where is version X of library Y deployed right now' in under five minutes? If not, your inventory does not work, regardless of what documents exist — and that is the thing to measure rather than SBOM coverage.

4 Responding to an advisory

- **1 · Determine exposure.** Which artefacts, which versions, and which of those are actually reachable — and 'present in the dependency tree' is not the same as 'reachable from your code', which is what reachability analysis tools exist to distinguish.
- **2 · Assess exploitability in your configuration.** Most advisories do not apply to most users — the vulnerable feature is not enabled, the input is not attacker-controlled, the component is not network-reachable — and **triaging on the published severity score alone wastes the budget.**
- **3 · Mitigate if you cannot patch immediately** — a configuration change, disabling the affected feature, or a filter in front of the vulnerable path. **A mitigation today beats a patch next week.**

- **4 · Patch, and then verify that the patched version is actually deployed everywhere — which is where responses quietly fail:** the fix is merged, one service is not redeployed, and the inventory still shows the old version six months later.
- **5 · And record what you decided and why,** because **the same library will have another advisory** and the analysis of whether the feature is enabled is reusable. **Step 2 is where judgement earns its keep:** a programme that patches every advisory at the same urgency will exhaust itself and miss the one that mattered.

THE HONEST ASSESSMENT OF DEPENDENCY SECURITY

You cannot review your dependency tree, and you cannot do without it — writing everything yourself trades a supply chain risk for a far larger quantity of your own bugs, which is a worse trade in almost every case.

So the realistic programme is: reduce the tree where you can, pin and verify, scan continuously, maintain an inventory you can actually query, harden the build (§2), and respond with judgement rather than by severity score (§4).

And accept the residual risk explicitly (Module 01 §2's fourth option) — **'we depend on 1,400 packages and review none of them'** is a true statement that belongs in the threat model rather than being left unsaid.

Which is more useful than a control nobody believes in. An accepted risk gets revisited when circumstances change, and an unacknowledged one does not — which is the whole argument for writing acceptances down.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
SLSA — Supply-chain Levels for Software Artifacts (free)	§2 and §3 as a graded framework — the levels are a usable roadmap rather than a checklist, and the build/release separation is explicit.
Ohm et al. — Backstabber's Knife Collection (free)	§1's risks, from real incidents — a systematic study of malicious package attacks and the mechanisms they used.
The Reproducible Builds project (free)	§3's strongest mechanism , with the practical guidance on the sources of non-determinism.
OpenSSF — Scorecard and the Concise Guides (free)	§1's adoption checks, automated — maintenance activity, review practices, and signing, scored per dependency.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Count your transitive dependencies** and compare against your direct ones.
- 2 **Find the three largest** and ask what fraction of each you actually use.
- 3 **List every package whose install scripts run** during your build.
- 4 **Disable install scripts** and report what breaks.
- 5 **Enumerate what credentials your build system holds**, and what each can reach.
- 6 **Check whether your build runs on ephemeral or persistent machines.**
- 7 **Generate an SBOM** and then time yourself answering 'where is library Y version X deployed'.
- 8 **Pick a past advisory** and run §4's five steps against your own system retrospectively.
- 9 **Find one advisory that does not apply to you** and write down why.
- 10 **Write your dependency risk acceptance** for the threat model.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Why is a dependency tree a security concern, and what is the trust decision really about?
- 2 Why are install scripts the highest-risk feature?
- 3 Name five dependency risks and the control for each.
- 4 Which risk has the fewest good controls, and why?
- 5 Why is the build system the highest-value target?
- 6 Name five build controls and say which is structural.
- 7 Name five provenance mechanisms and what each answers.
- 8 Why is reproducibility the strongest, and why is it hardest?
- 9 What is the real test of an SBOM?
- 10 Give the five-step advisory response and say where judgement matters.